RetailPOS

Microservices Platform

Interview-Ready Technical Walkthrough

Architecture: Microservices + Saga + RabbitMQ
Technology: .NET 8 | SQL Server | MassTransit | Ocelot
Services: ApiGateway, Identity, Admin, Catalog, Orders, Payment, Returns, Notification
Frontend: Angular 17 | SignalR Real-Time | RBAC Authorization

1. Project Overview — High-Level Narrative

What Problem Does This System Solve?

RetailPOS is a multi-store, multi-tenant retail Point-of-Sale platform built from the ground up using a Microservices architecture. The system solves real-world retail challenges:

- Traditional monolithic POS systems cannot be independently scaled — if the billing module crashes, the whole system goes down.
- Retail enterprises have multiple independent domains: Authentication, Product Catalog, Order Billing, Payment Collection, Stock Management, Returns, and Notifications — each requiring independent ownership and deployability.
- Distributed transactions (e.g., a checkout that spans Payment + Stock Deduction) need to be handled safely with compensation logic — solved here using the Saga Pattern via MassTransit.
- Real-time notifications (low stock, new order, new staff registration) are required at store level — solved via SignalR in the NotificationService.
- Enterprise RBAC (role-based access control) is required where each user role (Admin, Store Manager, Cashier) sees different menus and can perform different actions — solved using custom granular permission claims in JWT.

Architecture Type

This is a Microservices Architecture with the following key patterns in use:

Pattern	Technology Used	Where Applied
API Gateway	Ocelot (.NET)	ApiGateway project, port 5000
JWT Authentication	System.IdentityModel.Tokens.Jwt	IdentityService — AuthService.cs
Refresh Token	Secure HttpOnly Cookie	AuthController.cs → SetTokenCookie()
Saga Pattern	MassTransit StateMachine (EF Core)	OrdersService + ReturnsService Sagas
Message Broker	RabbitMQ via MassTransit	All services — Program.cs AddMassTransit
Repository Pattern	EF Core + GenericRepository<T>	All services — Repositories folder
Dependency Injection	ASP.NET Core DI (built-in)	All services — Program.cs registrations
SOLID Principles	Interface-based design (I* interfaces)	All services — Interfaces folder
Multi-Tenant (Store)	TenantProvider + JWT claims	OrdersService/Services/TenantProvider.cs
RBAC Authorization	Custom PermissionHandler + Policies	RetailPOS.Common/Authorization/
Real-Time Push	SignalR (NotificationHub)	NotificationService/Hubs/
Structured Logging	Serilog (file + console sink)	RetailPOS.Common/Logging/ — all services

ORM / DB Access	Entity Framework Core + SQL Server	All services — Data/ folder

Services Summary (8 Projects)

Service	Port	Responsibility
ApiGateway	5000	Central entry point — JWT validation + reverse proxy via Ocelot. Routes all client requests to downstream services.
IdentityService	5001	User registration, OTP email verification, two-factor login, JWT + Refresh Token generation, Google OAuth, RBAC seed.
CatalogService	5002	Product and Category CRUD with multi-tenant (per-store) filtering. Consumes stock adjustment & category events.
OrdersService	5003	Cart management, Bill creation, Bill finalization. Hosts the Checkout Saga (CheckoutStateMachine).
AdminService	5004	Store management, Staff assignment, Shift management, Reporting. Publishes StoreCreated events.
PaymentService	5005	Payment collection (POS cash/card) + Razorpay online payment. Consumes ProcessPaymentCommand from Saga.
ReturnsService	5006	Return initiation, Approval/Rejection, Saga-driven Refund + Restock flow (ReturnStateMachine).
NotificationService	5176	Consumes domain events (OrderPlaced, StockAdjusted, UserRegistered) and sends Email, SMS, and SignalR push.

Solution File: RetailPOS.slnx
Shared Library: RetailPOS.Common — Authorization, Logging
Shared Contracts: RetailPOS.Contracts — all MassTransit event/command interfaces
Frontend: pos-ui/ — Angular 17 SPA (npm start on port 4200)
Launch Script: run-all.ps1 — starts all 8 services + Angular in sequence

2. Project Structure Breakdown

Solution-Level Layout

```
RetailPOS.slnx
├── ApiGateway/              → Ocelot Gateway (port 5000)
├── IdentityService/         → Auth + JWT + RBAC (port 5001)
├── CatalogService/          → Products & Categories (port 5002)
├── OrdersService/           → Bills + Checkout Saga (port 5003)
├── AdminService/            → Stores + Staff (port 5004)
├── PaymentService/          → Payment Processing (port 5005)
├── ReturnsService/          → Returns + Refund Saga (port 5006)
├── NotificationService/     → Email + SMS + SignalR (port 5176)
├── RetailPOS.Common/        → Shared: Authorization, Logging
├── RetailPOS.Contracts/     → Shared: RabbitMQ Event/Command interfaces
├── pos-ui/                  → Angular 17 Frontend
└── run-all.ps1              → PowerShell launcher script
```

Internal Structure — Each Service

Every microservice follows the same layered structure. This is the standard layout:

```
[ServiceName]/
├── Controllers/         → HTTP entry points (REST API)
├── Services/            → Business logic layer
├── Repositories/        → Data access layer (GenericRepository<T>)
├── Interfaces/          → Abstractions for DI and SOLID compliance
├── Models/              → EF Core entity classes (DB tables)
├── DTOs/                → Data Transfer Objects (request/response shapes)
├── Consumers/           → MassTransit message consumers (RabbitMQ)
├── Sagas/               → MassTransit StateMachine (OrdersService, ReturnsService)
├── Data/                → EF Core DbContext + DbInitializer
├── Migrations/          → EF Core auto-generated migration files
├── Middleware/          → Exception handling middleware
├── Exceptions/          → Custom exception types
├── Program.cs           → DI registration + middleware pipeline
└── appsettings.json     → Connection strings, JWT config, RabbitMQ config
```

Shared Projects

Project	Purpose
RetailPOS.Common	Houses the enterprise RBAC system: Permissions.cs (all permission codes), PermissionHandler.cs (validates JWT claims), ServiceExtensions.cs (AddRetailPOSAuthorization() helper). Also has Logging/SerilogExtensions.cs (ConfigureSerilog()).
RetailPOS.Contracts	Contains all RabbitMQ message contracts as C# interfaces (CheckoutInitiatedEvent, OrderPlacedEvent, ProcessPaymentCommand, etc.). Referenced by all services to ensure type-safe messaging.

3. Application Startup Flow

Entry Point — Program.cs (all services use minimal hosting API)

All services follow the same startup pattern. Using IdentityService as the primary example:

Step 1 — Serilog Setup

```
builder.ConfigureSerilog("IdentityService");
```

File: RetailPOS.Common/Logging/SerilogExtensions.cs — configures structured logging to both Console and a rolling file under Logs/. Called before all other configuration.

Step 2 — MassTransit + RabbitMQ Registration

```
builder.Services.AddMassTransit(x => {
    x.AddConsumer<StoreCreatedConsumer>();
    x.AddConsumer<StaffAssignedConsumer>();
    x.UsingRabbitMq((context, cfg) => {
        cfg.Host("localhost", "/", h => { h.Username("guest"); h.Password("guest"); });
        cfg.ReceiveEndpoint("identity-store-created", e => { ... });
    });
});
```

This registers all RabbitMQ consumers and connects to the broker. MassTransit creates the queues automatically on first run. Orders/ReturnsService also add their StateMachines here using x.AddSagaStateMachine<...>().

Step 3 — Controller + JSON Config

```
builder.Services.AddControllers().AddJsonOptions(x => {
    x.JsonSerializerOptions.ReferenceHandler = ReferenceHandler.IgnoreCycles;
});
```

Step 4 — Swagger with Bearer Token Support

Each service adds Swagger with an AddSecurityDefinition("Bearer") so you can paste JWT tokens directly in the Swagger UI for testing.

Step 5 — JWT Authentication

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options => {
        options.TokenValidationParameters = new TokenValidationParameters {
            ValidIssuer = "RetailPOS",
            ValidAudiences = ["AdminService","CatalogService","OrdersService",...],
            IssuerSigningKey = new SymmetricSecurityKey(...)
        };
    });
```

The same JWT key and audiences are in every service's appsettings.json. A single token issued by IdentityService is valid across all downstream microservices.

Step 6 — Granular RBAC Authorization

```
builder.Services.AddRetailPOSAuthorization();
```

Defined in RetailPOS.Common/Authorization/ServiceExtensions.cs. This registers a PermissionHandler (IAuthorizationHandler) and creates one Authorization Policy per permission code (e.g., "orders:create", "catalog:manage"). Controllers then simply declare [Authorize(Policy = Permissions.Orders.Create)].

Step 7 — EF Core DbContext

```
builder.Services.AddDbContext<AppDbContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

Each service has its own independent database (e.g., RetailPOS_Identity, RetailPOS_Orders). This is the "database-per-service" microservices best practice implemented here.

Step 8 — Repository + Service DI

```
builder.Services.AddScoped<IUserRepository, UserRepository>();
builder.Services.AddScoped<IAuthService, AuthService>();
```

All dependencies are registered with Scoped lifetime, meaning one instance per HTTP request. This follows best practices for Entity Framework Core.

Step 9 — Middleware Pipeline (app.Build())

```
app.UseExceptionMiddleware();  // Custom: catches exceptions, returns JSON error
app.UseCors();                 // CORS (centralized in ApiGateway for most services)
app.UseAuthentication();       // Reads and validates JWT
app.UseAuthorization();        // Enforces permission policies
app.MapControllers();          // Routes requests to Controllers
app.Run();
```

Step 10 — Database Auto-Migration

On startup, IdentityService/OrdersService call context.Database.MigrateAsync() to auto-apply pending EF Core migrations. IdentityService also calls DbInitializer.InitAsync() to seed Roles, Permissions, and RolePermissions if they do not already exist.

4. End-to-End Request Flow

Flow A: Two-Factor Login (Most Important Flow)

Endpoint: POST /gateway/auth/login → IdentityService (port 5001)

Sample Request Body:
{ "email": "cashiers1@gmail.com", "password": "Test@1234" }

Expected Response (Step 1):
{ "requiresOtp": true, "message": "OTP_SENT" }

Step	Layer → File	What Happens
1	Ocelot ApiGateway ocelot.json	Gateway receives POST /gateway/auth/login. Matches UpstreamPathTemplate. Routes to http://localhost:5001/api/Auth/login. No AuthenticationOptions on this route (public endpoint).
2	Controller AuthController.cs → Login()	LoginDto is deserialized from request body. _authService.LoginAsync(loginDto) is called.
3	Service AuthService.cs → LoginAsync()	_userRepository.SingleOrDefaultAsync(u => u.Email == email) fetches user from DB. BCrypt.Verify() checks password hash. OTP (6-digit random) is generated. User.Otp and User.OtpExpiry saved. EmailService.SendEmailAsync() sends OTP (or logs for TestAccounts).
4	Repository UserRepository → GenericRepository<T>	EF Core query: SELECT * FROM Users WHERE Email = @email. SaveChangesAsync() persists OTP.
5	Response	200 OK: { requiresOtp: true, message: "OTP_SENT" }

Step 2 — OTP Verification: POST /gateway/auth/verify-login-otp

Request: { "email": "cashiers1@gmail.com", "otp": "123456" }

Response: { token: "eyJhbGci...", role: "Cashier", storeId: 1, permissions: ["orders:create", "orders:finalize", ...] }

Step	Layer → File	What Happens
1	Controller AuthController.cs → VerifyLoginOtp()	_authService.VerifyLoginOtpAsync(verifyDto) called.

2	Service AuthService.cs → VerifyLoginOtpAsync()	GetWithRolesByEmailAsync loads User with roles, stores, and permissions eagerly loaded. OTP + expiry validated. RefreshToken (64-byte random) generated and saved to DB.
3	JWT Generation AuthService → GenerateJwt()	Claims built: UserId, Email, Role, StoreId, ShiftDate, all Permission codes. JwtSecurityToken created with 1-hour expiry. Signed with HS256 and the shared secret key.
4	Cookie AuthController → SetTokenCookie()	Refresh token stored in HttpOnly Secure cookie "refreshToken" with 7-day expiry.
5	Response	Returns AuthResponseDto with JWT access token, role, storeId, storeCode, and permissions list.

Flow B: Create and Finalize a Bill (Core POS Flow + Saga Trigger)

This is the most architecturally complex flow, demonstrating the Checkout Saga.

Step 1 — Create Cart: POST /gateway/orders/bills

Header: Authorization: Bearer <jwt_token>
Request: { "billNumber": "B001", "customerMobile": "9999999999", "totalAmount": 500.00, "storeId": 1, "items": [{ "productId": 5, "quantity": 2, "unitPrice": 250.0, "subTotal": 500.0 }] }

Step	Layer → File	What Happens
1	Gateway ocelot.json	Routes /gateway/orders/bills → http://localhost:5003/api/Bills. JWT validated at gateway level.
2	Controller BillsController.cs → Create()	[Authorize(Policy = Permissions.Orders.Create)] enforced. _billService.CreateOrUpdateCartAsync(billDto) called.
3	Service OrderServices.cs → CreateOrUpdateCartAsync()	New Bill entity created with Status="Draft". TenantProvider.StoreId reads StoreId from JWT claim if not provided. BillItems added. SaveChangesAsync() persists.
4	Response	200 OK with created Bill DTO including generated Bill ID (e.g., 42).

Step 2 — Finalize Bill (Triggers Saga): POST /gateway/orders/bills/42/finalize

Step	Layer → File	What Happens
1	Controller BillsController.cs → Finalize()	[Authorize(Policy = Permissions.Orders.Finalize)] checked.

2	Service OrderServices.cs → FinalizeBillAsync()	Bill status updated to "PendingPayment". IPublishEndpoint.Publish<CheckoutInitiatedEvent>() publishes event to RabbitMQ with OrderId, TotalAmount, Items.
3	Saga (Created) CheckoutStateMachine.cs	MassTransit receives CheckoutInitiatedEvent. NEW saga instance created in OrdersSagaDb (CheckoutSagaState table). Saga state = "PaymentProcessing". PublishAsync<ProcessPaymentCommand> sent to RabbitMQ.
4	PaymentService Consumer CheckoutSagaCommandsConsumer.cs → Consume(ProcessPaymentCommand)	Payment record created in PaymentDb. PaymentProcessedEvent published back to RabbitMQ with Status="Success".
5	Saga (Continues) CheckoutStateMachine.cs → During(PaymentProcessing)	PaymentProcessed received. Payment.Status=="Success" → transition to "StockDeductionProcessing". DeductStockCommand published to RabbitMQ.
6	CatalogService Consumer SagaDeductStockConsumer.cs	Receives DeductStockCommand. Reduces Product.Quantity in CatalogDb. Publishes StockDeductedEvent.
7	Saga (Completes) CheckoutStateMachine.cs → During(StockDeductionProcessing)	StockDeducted received. FinalizeOrderCommand + OrderPlacedEvent published. Saga state = "Completed". Finalized.
8	OrdersService Consumer SagaOrderCommandsConsumer.cs	Receives FinalizeOrderCommand. Bill.Status updated to "Finalized" in DB. OrderPlacedEvent published to RabbitMQ.
9	NotificationService Consumer OrderPlacedConsumer.cs	Receives OrderPlacedEvent. SMS sent to customer. SignalR push to store dashboard.

Compensation Path (if Stock Deduction Fails)

If CatalogService publishes StockDeductionFailedEvent instead:

- Saga transitions to "Refunding" state.
- RefundPaymentCommand published to PaymentService.
- PaymentService publishes PaymentRefundedEvent.
- Saga transitions to "Failed" and finalizes.

This is the Saga compensating transaction in action — ensuring distributed consistency without a 2-phase commit.

5. Code-Level Deep Dive

5.1 Repository Pattern — GenericRepository<T>

File: IdentityService/Repositories/GenericRepository.cs (same pattern in all services)

This is a generic data access layer that wraps Entity Framework Core. All specific repositories (UserRepository, BillRepository, etc.) inherit from this class.

Method	What It Does
GetByIdAsync(int id)	Uses EF Core _dbSet.FindAsync(id) — primary key lookup by ID.
GetAllAsync()	_dbSet.ToListAsync() — retrieves all rows for the entity type.
FindAsync(Expression predicate)	WHERE clause query using LINQ expression tree.
SingleOrDefaultAsync(predicate)	Returns one result or null — used for email lookups.
AddAsync(T entity)	_dbSet.AddAsync — stages entity for INSERT.
Update(T entity)	Sets EntityState.Modified — stages entity for UPDATE.
SaveChangesAsync()	Commits the EF Core Unit of Work — executes pending SQL.

5.2 TenantProvider — Multi-Store Context Isolation

File: OrdersService/Services/TenantProvider.cs (same pattern in other services)

This is a critical piece for multi-store isolation. It reads the StoreId from the JWT token's claims so each cashier only sees data for their own store.

```
public int StoreId {
    get {
        // Admin can override via X-Store-Id header
        if (role == "Admin" && Request.Headers["X-Store-Id"] exists) → return it
        // Otherwise read from JWT claim
        var claim = context.User.FindFirst("StoreId");
        return int.Parse(claim.Value);
    }
}
```

The DbContext uses this StoreId in a global query filter: modelBuilder.Entity<Bill>().HasQueryFilter(b => b.StoreId == _tenantProvider.StoreId), ensuring data isolation automatically.

5.3 RBAC Authorization — PermissionHandler

File: RetailPOS.Common/Authorization/PermissionHandler.cs

This custom IAuthorizationHandler is the core of the enterprise RBAC system.

```
protected override Task HandleRequirementAsync(context, requirement) {
    // Super Admin shortcut
    if (user.HasClaim("permission", "all:all")) { context.Succeed(); return; }
    // Check specific granular permission
    if (user.HasClaim("permission", requirement.Permission)) { context.Succeed(); }
```

```
}
```

When JWT is generated in AuthService.GenerateJwt(), all permission codes (e.g., "orders:create", "catalog:manage") are embedded as individual "permission" claims. Each Authorization Policy maps to one permission code. A controller method with [Authorize(Policy = "orders:finalize")] will ONLY be accessible to users who have that exact claim in their token.

5.4 Checkout Saga State Machine

File: OrdersService/Sagas/CheckoutStateMachine.cs

MassTransit StateMachine orchestrates the distributed checkout transaction:

State	Meaning
Initial	No saga started yet.
PaymentProcessing	Waiting for PaymentService to confirm payment.
StockDeductionProcessing	Payment done; waiting for CatalogService to deduct stock.
Refunding	Stock failed; waiting for PaymentService to issue refund.
Completed	Everything succeeded. Saga finalized.
Failed	Payment or stock deduction failed. Saga finalized.

Saga state is persisted to SQL Server (OrdersSagaDb, table CheckoutSagaState) by MassTransit's EntityFrameworkRepository. This means if the service restarts mid-transaction, the saga picks up from where it left off.

5.5 SOLID Principles — Where They Appear in Code

Principle	Where in Code	Example
S — Single Responsibility	Each class has one job	AuthService.cs handles only authentication. EmailService.cs handles only email. PaymentService handles only payment.
O — Open/Closed	NotificationService consumers	New notification types (SMS, Email, SignalR) added by implementing INotificationService without changing existing consumers.
L — Liskov Substitution	GenericRepository<T>	UserRepository, BillRepository, PaymentRepository all extend GenericRepository<T> and can be substituted without breaking behavior.
I — Interface Segregation	Interfaces/ folder in every service	IAuthService, IEmailService, IBillService — each has its own focused interface. Not one giant IService.
D — Dependency Inversion	All constructors use interfaces	BillService depends on IBillRepository (not BillRepository). AuthService depends on

		IUserRepository (not UserRepository). Wired in Program.cs DI.

6. Database Design & EF Core Interaction

Database-Per-Service (Microservices Best Practice)

Each service owns its own SQL Server database. This is enforced via separate ConnectionStrings in each service's appsettings.json.

Service	Database Name
IdentityService	RetailPOS_Identity (Users, Roles, Permissions, Stores, UserStoreRoles)
CatalogService	RetailPOS_Catalog (Products, Categories — with StoreId filter)
OrdersService	RetailPOS_Orders (Bills, BillItems, Customers) + RetailPOS_OrdersSaga (Saga state)
AdminService	RetailPOS_Admin (Stores, Staff, Shifts, Reports)
PaymentService	RetailPOS_Payments (Payments)
ReturnsService	RetailPOS_Returns (Returns) + RetailPOS_ReturnsSaga (Saga state)
NotificationService	RetailPOS_Notifications (Notifications log)

EF Core Usage Pattern

All services use Code-First migrations. Entity configuration happens in DbContext's OnModelCreating() method.

Key patterns used:

- Global Query Filters: modelBuilder.Entity<Bill>().HasQueryFilter(b => b.StoreId == _tenantProvider.StoreId) — ensures store-level data isolation automatically on every query.
- Navigation Properties: Bill has ICollection<BillItem> Items. EF Core loads these with Include() in repository queries.
- EF Core Migrations: Run automatically on service startup via context.Database.MigrateAsync() in Program.cs.
- Saga DB Context: OrdersService has two DbContexts — OrdersDbContext (application data) and OrdersSagaDbContext (MassTransit saga state tables).

Key Entities and Their Relationships

Entity	Service	Key Fields
User	IdentityService	Id, Email, PasswordHash, IsEmailVerified, Otp, RefreshToken, UserRoles (nav)
UserStoreRole	IdentityService	UserId, StoreId, RoleId — the M:N mapping between User, Store, and Role

Role → RolePermission → Permission	IdentityService	Hierarchical RBAC: Role has many RolePermissions; each maps to a Permission.Code like "orders:create"
Bill	OrdersService	Id, BillNumber, Status, StoreId, TotalAmount, Items (nav), CustomerMobile
CheckoutSagaState	OrdersService (Saga DB)	CorrelationId (Guid), CurrentState, OrderId, TotalAmount, PaymentId — MassTransit saga instance
Payment	PaymentService	Id, BillId, Amount, PaymentMode, ReferenceNumber, StoreId
Product / Category	CatalogService	Id, Name, Price, StockQuantity, StoreId, IsActive (soft delete)
Return / ReturnSagaState	ReturnsService	Id, OrderId, Status, StoreId — plus ReturnSagaState for the MassTransit state machine

7. Async / Messaging Flow (RabbitMQ + MassTransit)

Full Checkout Event Flow (Producer → Queue → Consumer)

Message contracts are defined in RetailPOS.Contracts project as C# interfaces.
MassTransit automatically creates RabbitMQ exchanges and queues based on message type names.
Services reference these contracts — ensuring compile-time safety on all messages.

Event / Command	Producer	Consumer(s)
CheckoutInitiatedEvent	OrdersService/BillService.cs → FinalizeBillAsync()	OrdersService/CheckoutStateMachine (Saga)
ProcessPaymentCommand	CheckoutStateMachine (Saga — PaymentProcessing state)	PaymentService/CheckoutSagaCommandsConsumer.cs
PaymentProcessedEvent	PaymentService/CheckoutSagaCommandsConsumer.cs	CheckoutStateMachine (Saga continues)
DeductStockCommand	CheckoutStateMachine (after payment success)	CatalogService/SagaDeductStockConsumer.cs
StockDeductedEvent	CatalogService/SagaDeductStockConsumer.cs	CheckoutStateMachine (Saga continues)
StockDeductionFailedEvent	CatalogService/SagaDeductStockConsumer.cs	CheckoutStateMachine → triggers Refund
RefundPaymentCommand	CheckoutStateMachine (compensation)	PaymentService/CheckoutSagaCommandsConsumer.cs
PaymentRefundedEvent	PaymentService consumer	CheckoutStateMachine (Saga → Failed state)
FinalizeOrderCommand	CheckoutStateMachine (after StockDeducted)	OrdersService/SagaOrderCommandsConsumer.cs → sets Bill.Status = "Finalized"
OrderPlacedEvent	OrdersService/SagaOrderCommandsConsumer.cs	NotificationService/OrderPlacedConsumer.cs → SMS + SignalR push
UserRegisteredEvent	IdentityService/AuthService → RegisterAsync()	NotificationService/UserRegisteredConsumer.cs → Admin email alert
StoreCreatedEvent	AdminService (store creation)	IdentityService/StoreCreatedConsumer.cs → syncs store for login
ReturnInitiatedEvent	ReturnsService (return creation)	ReturnsService/ReturnStateMachine (Saga start), NotificationService

RabbitMQ Best Practices Applied in This Project

- Idempotent Consumers: StoreCreatedConsumer checks if store already exists before inserting — prevents duplicate data on redelivery.

- CorrelateBy on OrderId: Saga uses CorrelateBy((saga, ctx) => saga.OrderId == ctx.Message.OrderId) to match messages to the right saga instance without requiring a global ID.
- ConfigureEndpoints(context): OrdersService and PaymentService use MassTransit's automatic endpoint configuration — queues named by convention from consumer class names.
- Manual Endpoints (IdentityService): ReceiveEndpoint("identity-store-created", ...) — explicitly named queues for specific cross-service event subscriptions.
- StateMachine + EF Core Persistence: Saga state stored in SQL Server so it survives service restarts — true reliability pattern.

8. Execution Demo Guide (Critical for Interview)

Prerequisites

- SQL Server (Express or Developer edition) running on localhost\SQLEXPRESS
- RabbitMQ running on localhost:5672 (Management UI: http://localhost:15672 — user: guest/guest)
- .NET 8 SDK installed
- Node.js 18+ and Angular CLI for the frontend (optional — backend alone is sufficient for demo)

How to Start All Services

Option A — Automated (Recommended):

```
cd Project/
.\run-all.ps1
```

This PowerShell script stops any existing dotnet processes and launches all 8 services + Angular in separate windows with a 5-second delay between each.

Option B — Manual (Individual Terminal):

```
dotnet run --project IdentityService/IdentityService.csproj   --urls
http://127.0.0.1:5001
dotnet run --project AdminService/AdminService.csproj          --urls
http://127.0.0.1:5004
dotnet run --project CatalogService/CatalogService.csproj      --urls
http://127.0.0.1:5002
dotnet run --project OrdersService/OrdersService.csproj        --urls
http://127.0.0.1:5003
dotnet run --project PaymentService/PaymentService.csproj      --urls
http://127.0.0.1:5005
dotnet run --project ReturnsService/ReturnsService.csproj      --urls
http://127.0.0.1:5006
dotnet run --project NotificationService/NotificationService.csproj --urls
http://127.0.0.1:5176
dotnet run --project ApiGateway/ApiGateway.csproj              --urls
http://127.0.0.1:5000
```

Swagger UI Access URLs

Service	Swagger URL
ApiGateway	http://localhost:5000 (no swagger — Ocelot proxy)
IdentityService	http://localhost:5001/swagger
CatalogService	http://localhost:5002/swagger
OrdersService	http://localhost:5003/swagger
AdminService	http://localhost:5004/swagger
PaymentService	http://localhost:5005/swagger
ReturnsService	http://localhost:5006/swagger

NotificationService	http://localhost:5176/swagger
Angular UI	http://localhost:4200
RabbitMQ Management	http://localhost:15672

Recommended Demo Sequence

STEP 1 — Show Architecture (talk for 2 minutes while showing the diagram)

1. Open http://localhost:5000 — show "Ocelot API Gateway is running."
2. Open RabbitMQ at http://localhost:15672 — show queues/exchanges. Explain all services are connected.

STEP 2 — Login Flow (IdentityService Swagger: http://localhost:5001/swagger)

3. POST /api/Auth/login with { "email": "cashiers1@gmail.com", "password": "Test@1234" }
4. Show "OTP_SENT" response. Explain: "We do not give a token yet — 2FA is enforced."
5. Check service logs/console for "[TEST OTP] Login OTP for cashiers1@gmail.com: 123456"
6. POST /api/Auth/verify-login-otp with the OTP — show full token response with permissions array.
7. Copy the token. In Swagger, click Authorize, paste the token. Explain JWT structure.

STEP 3 — Show RBAC (OrdersService Swagger: http://localhost:5003/swagger)

8. Without token: GET /api/Bills → 401 Unauthorized. "No token = no access."
9. With cashier token: GET /api/Bills → 200 (only this store's bills). "Role + StoreId baked into token."
10. Try DELETE (if not in cashier permissions) → 403 Forbidden. "Granular permission policy enforced."

STEP 4 — Create Bill and Trigger Checkout Saga

11. POST /api/Bills — create a draft bill with items. Show returned Bill ID.
12. POST /api/Bills/{id}/finalize — trigger the saga. Watch all service consoles light up in sequence.
13. Check RabbitMQ management — show the message flow through queues.
14. Check OrdersService DB — bill status changed from Draft → PendingPayment → Finalized.
15. Check PaymentService DB — payment record created automatically by the saga.
16. Check CatalogService DB — product stock quantity reduced.

STEP 5 — Show Notification (if SignalR demo is ready)

17. Check NotificationService logs for "ORDER RECEIVED: New bill #X finalized for ₹Y."

Key Sample Request Payloads

Login (Step 1):

```
POST http://localhost:5001/api/Auth/login
{ "email": "cashiers1@gmail.com", "password": "Test@1234" }
```

OTP Verify (Step 2):

```
POST http://localhost:5001/api/Auth/verify-login-otp
{ "email": "cashiers1@gmail.com", "otp": "123456" }
```

Create Bill (Cart):

```
POST http://localhost:5003/api/Bills
Authorization: Bearer <token>
{
  "billNumber": "BILL-001",
  "customerMobile": "9999988888",
  "customerName": "Test Customer",
  "totalAmount": 500.00,
  "taxAmount": 25.00,
  "storeId": 1,
  "items": [{ "productId": 1, "productName": "Item A", "quantity": 2, "unitPrice": 250.0,
"subTotal": 500.0 }]
}
```

Finalize Bill (Triggers Saga):

```
POST http://localhost:5003/api/Bills/{id}/finalize
Authorization: Bearer <token>
```

9. Simplified Explanation Script (Spoken for Interview)

"Can you walk me through your project?"

"We built RetailPOS — a multi-store Point-of-Sale platform using a Microservices architecture on .NET 8.

The system is split into 8 independently deployable services — an API Gateway using Ocelot, an Identity Service for JWT-based two-factor authentication, a Catalog Service for product management, an Orders Service for billing, a Payment Service, a Returns Service, an Admin Service for store and staff management, and a Notification Service for real-time push alerts.

All services communicate through RabbitMQ using MassTransit. For the core checkout flow, we implemented the Saga Pattern — specifically a Choreography-style Saga using MassTransit StateMachine. When a cashier finalizes a bill, the Saga coordinates payment processing and stock deduction across three independent services. If stock deduction fails after payment succeeds, the Saga automatically triggers a refund — that's the compensating transaction.",

"How does authentication work?"

"We use a two-factor JWT authentication flow.

Step 1: User submits email and password. IdentityService verifies the BCrypt hash, generates a 6-digit OTP, and sends it via email (using Gmail SMTP). The response tells the client to expect an OTP — no token is issued yet.

Step 2: User submits the OTP. IdentityService validates it, loads the user's role and all their granular permissions from the database, and bakes those permission codes directly into the JWT as custom claims — for example, orders:create, orders:finalize, catalog:view.",

The JWT is valid for 1 hour. We also issue a Refresh Token (64-byte random value) stored in an HttpOnly Secure cookie, valid for 7 days. Services validate the token using the same shared secret key and audience list."

"How does the Saga pattern work in your project?"

"In a microservices system, a single business transaction — like a retail checkout — spans multiple services. You cannot use a database transaction across service boundaries. That's the distributed transaction problem.

The Saga Pattern solves this by breaking one big transaction into smaller local transactions, each completed by one service. If any step fails, compensating transactions are triggered to undo the previous steps.

In our checkout flow: the Bill is finalized → CheckoutInitiatedEvent published → CheckoutStateMachine (Saga) starts → sends ProcessPaymentCommand to PaymentService → on success, sends DeductStockCommand to CatalogService → on success, FinalizeOrderCommand sent back to OrdersService and OrderPlacedEvent published for notifications.

The saga state is stored in SQL Server via MassTransit's EntityFrameworkRepository — so if the service crashes mid-transaction, it picks up from where it left off."

"Tell me about SOLID in your project."

"We applied all five SOLID principles throughout the codebase.

Single Responsibility: Each class has one job. AuthService only handles authentication. EmailService only sends emails. TenantProvider only resolves the current store context.

Open/Closed: The notification system uses an INotificationService interface. Adding a new notification channel (Slack, WhatsApp) requires only a new implementation — no changes to existing consumers.

Liskov Substitution: All repositories extend GenericRepository<T> and implement their specific interface. They can be swapped without breaking the service layer.

Interface Segregation: Every service has focused interfaces — IAuthService, IBillService, IPaymentService. There is no god interface.

Dependency Inversion: Every class depends on an interface, not a concrete type. All wiring happens in Program.cs DI registrations, not inside the classes themselves."

"How do you handle multi-tenancy?"

"Each user's JWT token contains their StoreId as a claim. When a cashier logs into Store A, their token says StoreId: 1.

The TenantProvider (injected via IHttpContextAccessor) reads this claim on every request. The EF Core DbContext uses a Global Query Filter that automatically appends WHERE StoreId = {currentStoreId} to every query — so a cashier at Store A can never see Store B's orders, products, or customers.

Admin users can pass an X-Store-Id header to override this and view any store's data — this is also handled in TenantProvider."

"What best practices did you follow?"

1. Database-per-service — each microservice owns its own SQL Server database.
2. Centralized API Gateway — all client requests go through Ocelot (port 5000) which validates JWT and proxies to services.
3. Shared JWT secret — one token issued by IdentityService is valid across all 7 downstream services.
4. Idempotent message consumers — checked before inserting (e.g., StoreCreatedConsumer checks if store exists).
5. Saga-driven distributed transactions — no 2-phase commits; compensating actions on failure.
6. Granular RBAC — permission-level authorization, not just role-level.
7. Structured logging with Serilog — correlation IDs, file + console sinks, all services.
8. DTO separation — Models (EF entities) never exposed directly; DTOs used for request/response.
9. Repository pattern — business logic never directly touches DbContext.
10. RefreshToken with HttpOnly Cookie — secure token rotation implemented.

10. Quick Reference — File Map for Interview Questions

If Asked About...

Topic / Question	Exact File(s) to Point To
JWT generation	IdentityService/Services/AuthService.cs → GenerateJwt() method
OTP two-factor login	IdentityService/Services/AuthService.cs → LoginAsync() + VerifyLoginOtpAsync()
Refresh token	IdentityService/Services/AuthService.cs → RefreshTokenAsync(), AuthController.cs → Refresh()
Google OAuth	IdentityService/Services/AuthService.cs → GoogleLoginAsync()
RBAC / Permission system	RetailPOS.Common/Authorization/PermissionHandler.cs + Permissions.cs
Checkout Saga	OrdersService/Sagas/CheckoutStateMachine.cs + CheckoutSagaState.cs
Returns Saga	ReturnsService/Sagas/ReturnStateMachine.cs + ReturnSagaState.cs
RabbitMQ contracts	RetailPOS.Contracts/ — all .cs files
Payment consumer	PaymentService/Consumers/CheckoutSagaCommandsConsumer.cs
Stock deduction consumer	CatalogService/Consumers/SagaDeductStockConsumer.cs
Order finalization consumer	OrdersService/Consumers/SagaOrderCommandsConsumer.cs
Notification dispatch	NotificationService/Consumers/EventConsumers.cs
Multi-tenancy / Store isolation	OrdersService/Services/TenantProvider.cs
API Gateway routing	ApiGateway/ocelot.json + ApiGateway/Program.cs
Startup / DI wiring	Any service's Program.cs
Database migrations	Any service's Migrations/ folder
Generic repository	IdentityService/Repositories/GenericRepository.cs (template for all services)
Exception middleware	Any service's Middleware/ folder (ExceptionMiddleware.cs)
Structured logging	RetailPOS.Common/Logging/SerilogExtensions.cs
Store sync (cross-service)	IdentityService/Consumers/StoreCreatedConsumer.cs
How to run the project	Project/run-all.ps1